

**UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS**  
**(Universidad del Perú, DECANA DE AMÉRICA)**  
**FACULTAD DE INGENIERÍA ELECTRÓNICA Y ELÉCTRICA**



**DOCENTE:**

Villafuerte Barreto, Hernan Oswaldo

**CURSO:**

Circuitos Digitales

**INTEGRANTES:**

Melendez Romero Jose Francesco

Sabrera Ortiz, Angie Roxana

Berrocal Casanca Esaú Andree

**LIMA - PERÚ**

**2026**

# **TÍTULO: Implementación de un sistema digital de validación de integridad por paridad longitudinal con banco de registros direccionables.**

## **1. Planteamiento del Problema**

En la transmisión de datos digitales, el ruido en el canal puede alterar uno o más bits de una trama, generando errores silenciosos en la información recibida. En sistemas de telemetría de baja velocidad, es crítico contar con un módulo de hardware dedicado que verifique la integridad de cada trama mediante un algoritmo de detección de errores, y que además registre los resultados para una auditoría posterior sin intervención de software. Este proyecto propone el diseño e implementación, en Logisim Evolution, de un sistema digital que recibe tramas de 8 bits con checksum adjunto, calcula la verificación mediante XOR bit a bit en lógica combinacional, y almacena hasta 4 registros validados en un banco de flip-flops D direccionado por un contador de 2 bits. El sistema emite además una señal de bandera de error ante inconsistencias detectadas, simulando el comportamiento de la capa de enlace presente en protocolos como UART o I2C.

## **2. Objetivos**

### **1. Objetivo General:**

Diseñar e implementar un sistema digital de verificación de integridad de datos basado en Checksum XOR, que permita detectar errores en tramas de 8 bits y almacenar los resultados en un banco de registros direccionables, utilizando lógica combinacional y secuencial en un entorno de simulación digital.

### **2. Objetivos Específicos:**

- Diseñar el bloque de verificación de integridad mediante operaciones XOR bit a bit, capaz de calcular y comparar el checksum de cada trama recibida.
- Implementar un banco de registros de 4 posiciones utilizando flip-flops tipo D, para el almacenamiento de las tramas validadas.
- Desarrollar un sistema de direccionamiento basado en un contador de 2 bits y un decodificador, que permita seleccionar correctamente cada posición del banco de registros.
- Diseñar una máquina de estados finitos (FSM) que controle el flujo de recepción, verificación, almacenamiento y generación de la señal de error.
- Evaluar el desempeño del sistema mediante métricas como tiempo de respuesta, tasa de detección de errores y utilización de recursos lógicos en la simulación

## **3. Justificación Técnica y Alineación con la Sumilla**

El presente proyecto se fundamenta en el diseño e implementación de un sistema digital orientado a la verificación de integridad de datos en transmisiones digitales, una función esencial dentro de la capa de enlace de datos en sistemas de telecomunicaciones. La detección de errores mediante operaciones lógicas, como el Checksum basado en XOR, constituye una solución eficiente y de bajo costo computacional para identificar alteraciones en tramas de datos causadas por ruido en el canal.

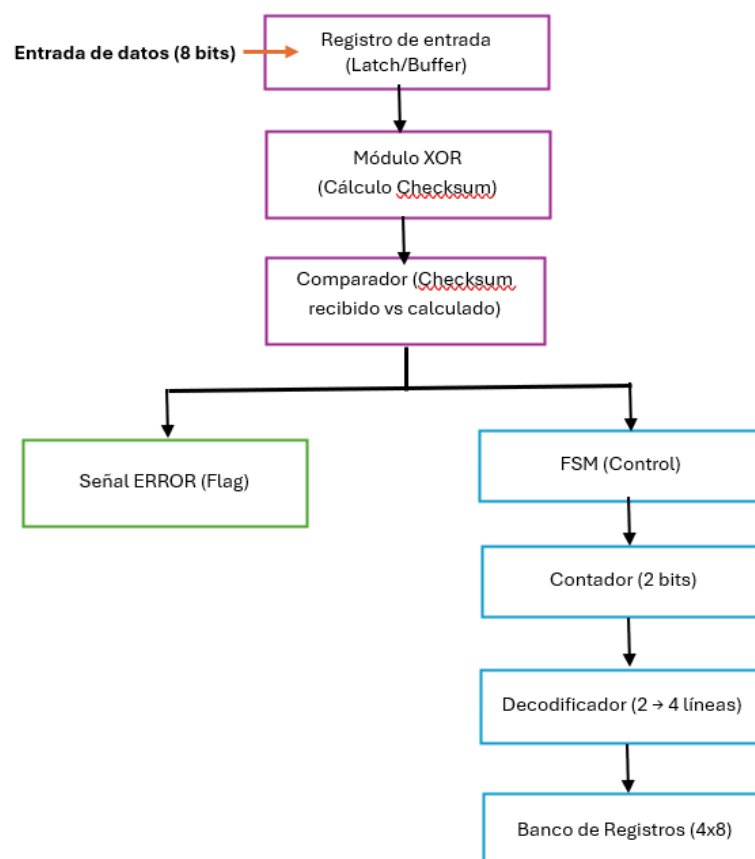
Desde el punto de vista técnico, el sistema integra conceptos clave de los sistemas digitales. En primer lugar, se emplea **lógica combinacional** para el cálculo del checksum mediante operaciones XOR bit a bit, lo que permite comparar en tiempo real los datos recibidos con el valor esperado. En segundo lugar, se incorpora **lógica secuencial** mediante el uso de flip-flops tipo D y una máquina de estados finitos (FSM), encargada de controlar las etapas de recepción, verificación, almacenamiento y señalización de error.

Asimismo, el diseño incluye un **banco de registros de 4 posiciones**, que actúa como memoria del sistema, permitiendo almacenar temporalmente las tramas procesadas. El acceso a esta memoria se realiza mediante un **decodificador de direcciones**, controlado por un contador de 2 bits, el cual selecciona de manera determinística la posición de almacenamiento correspondiente. Esta interacción entre FSM, decodificador y memoria evidencia la estructura de un subsistema de procesamiento digital, tal como lo exige la sumilla del curso.

En cuanto a su alineación con la asignatura de Sistemas Digitales, el proyecto cumple con los requerimientos establecidos al integrar de manera práctica los fundamentos de análisis y diseño de circuitos combinacionales y secuenciales, así como el uso de circuitos de decodificación de memoria dentro de una arquitectura funcional. Además, el uso de herramientas de simulación como Logisim Evolution permite validar el comportamiento del sistema y analizar métricas de desempeño relevantes.

Finalmente, este proyecto no solo refuerza los conocimientos teóricos del curso, sino que también los contextualiza en una aplicación real de telecomunicaciones, aportando una solución concreta para la detección de errores en la transmisión de datos digitales.

#### 4. Diagrama de Bloques Funcional de Alto Nivel



1. El sistema recibe la trama (**DATA\_IN**) y su checksum (**CHECKSUM\_IN**).
2. El módulo XOR calcula el checksum esperado.
3. El comparador verifica si ambos coinciden.
4. La FSM evalúa el resultado:
  - Si no hay error → se almacena en memoria.
  - Si hay error → se activa **ERROR\_FLAG**.
5. El contador genera la dirección de almacenamiento.
6. El decodificador habilita el registro correspondiente.

## 5. Selección Justificada de Plataforma

El sistema será implementado sobre la plataforma FPGA DE2-115 (Intel/Altera Cyclone IV EP4CE115F29C7), utilizando el entorno de desarrollo Quartus Prime Lite para la síntesis, simulación funcional y análisis de temporización. Esta elección se justifica por las siguientes razones técnicas:

En primer lugar, la FPGA Cyclone IV EP4CE115 dispone de una gran cantidad de elementos lógicos (LEs), bloques de memoria embebida (M9K) y registros, lo que permite implementar sin limitaciones la máquina de estados finitos (FSM), el banco de registros de 4 posiciones y el bloque combinacional de verificación XOR. Estos recursos son adecuados para desarrollar sistemas digitales que integren lógica combinacional, secuencial y memoria, en concordancia con los requerimientos del curso.

En segundo lugar, la arquitectura de las FPGA permite ejecutar operaciones en paralelo, lo cual resulta ideal para el cálculo del checksum y la verificación de integridad en tiempo real. A diferencia de los microcontroladores, donde las operaciones se ejecutan de manera secuencial, el uso de FPGA garantiza mayor eficiencia temporal y control a nivel de hardware. Asimismo, la tarjeta DE2-115 incorpora un oscilador de 50 MHz, interruptores, LEDs y displays de 7 segmentos integrados, los cuales facilitan la implementación de interfaces de entrada y salida del sistema, permitiendo visualizar el estado de las tramas y la señal de error sin requerir hardware adicional.

Adicionalmente, el entorno Quartus Prime permite obtener reportes detallados de análisis de temporización (frecuencia máxima de operación, retardos, slack) y utilización de recursos (LUTs, registros, memoria), lo cual resulta fundamental para la evaluación de métricas de desempeño del sistema. Finalmente, como herramienta de verificación preliminar, se empleará Logisim Evolution para validar el comportamiento lógico del diseño a nivel conceptual antes de su implementación en HDL, reduciendo así la probabilidad de errores en etapas posteriores del desarrollo.

## 6. Listado Preliminar de Métricas de Rendimiento

Se definieron seis métricas cuantitativas que permitirán evaluar de forma objetiva el desempeño del sistema. Para cada una se indica el método de medición y el criterio de aceptación:

### - **Latencia de verificación (T\_verify):**

Tiempo transcurrido desde que el último bit de la trama ingresa al sistema hasta que la señal **Integrity\_Flag** se estabiliza en su valor definitivo. Se medirá mediante simulación funcional y

análisis de formas de onda (ModelSim), considerando opcionalmente simulación post-síntesis.

Criterio de aceptación:  $T_{\text{verify}} \leq 2$  ciclos de reloj a 50 MHz ( $\leq 40$  ns).

- **Latencia de escritura en banco de registros ( $T_{\text{write}}$ ):**

Tiempo desde la activación de la señal de escritura por la FSM hasta que el dato queda estable en la posición de memoria seleccionada. Se medirá mediante simulación funcional.

Criterio de aceptación:  $T_{\text{write}} \leq 1$  ciclo de reloj ( $\leq 20$  ns).

- **Frecuencia máxima de operación ( $F_{\text{max}}$ ):**

Frecuencia máxima a la que el sistema puede operar sin violar restricciones de temporización (setup/hold), determinada a partir del análisis del camino crítico utilizando la herramienta TimeQuest Timing Analyzer de Quartus.

Criterio de aceptación:  $F_{\text{max}} \geq 50$  MHz.

- **Utilización de recursos lógicos:**

Cantidad de elementos lógicos (LEs), flip-flops y bloques de memoria utilizados tras la síntesis, expresados como porcentaje del total disponible en el dispositivo. Criterio de aceptación: Uso reducido de recursos ( $<5\%$ ), evidenciando un diseño eficiente y escalable,

- **Tasa de detección de errores (TDE):**

Porcentaje de tramas con errores detectadas correctamente sobre un conjunto de pruebas con fallas inducidas (1 y 2 bits alterados).

Criterio de aceptación:

- 100% para errores de 1 bit
- $\geq 50\%$  para errores de 2 bits (limitación inherente del método XOR)

- **Cobertura funcional de la FSM:**

Porcentaje de transiciones de estado ejercitadas durante la simulación mediante un testbench, verificando la correcta operación del sistema en todos los escenarios definidos.

Criterio de aceptación: Cobertura  $\geq 90\%$ .

## 7. Cronograma Interno de Trabajo

| Sema     | Actividad Principal                | Descripción de tareas                                                                    | Entregable / Resultado            |
|----------|------------------------------------|------------------------------------------------------------------------------------------|-----------------------------------|
| 4        | Definición del proyecto            | Selección del tema, planteamiento del problema, objetivos y                              | Idea del proyecto definida        |
| 5 a la 6 | Elaboración de propuesta (E1)      | Redacción del documento: problema, objetivos, diagrama de bloques, métricas y plataforma | Entrega E1                        |
| 7        | Ajustes de propuesta               | Correcciones según retroalimentación del docente                                         | Propuesta validada                |
| 8        | Diseño detallado + simulación (E2) | Diseño de FSM, decodificador, memoria; desarrollo en Logisim/HDL;                        | Entrega E2 + simulación funcional |
| 9        | Optimización del diseño            | Corrección de errores, mejora de FSM y lógica combinacional                              | Sistema funcional en simulación   |
| 10       | Preparación de implementación      | Adaptación del diseño a FPGA (codificación en Verilog/VHDL)                              | Código listo para síntesis        |
| 11       | Implementación en hardware (E3)    | Carga en FPGA, pruebas básicas, medición inicial de métricas                             | Entrega E3 + video corto          |
| 12       | Ajustes y mejoras                  | Corrección de errores en hardware, optimización de timing                                | Sistema estable en hardware       |
| 13       | Sistema completo + métricas (E4)   | Medición completa de métricas, validación del sistema, inicio del                        | Entrega E4 + borrador de paper    |
| 14       | Redacción final del paper          | Corrección del paper, análisis de resultados, conclusiones                               | Paper casi finalizado             |
| 15       | Entrega final (E5)                 | Presentación final, video demostrativo, entrega del paper IEEE                           | Entrega E5 + exposición           |

## 8. Referencias

Computer Science Wiki. (s.f.). *Checksums*. Recuperado de <https://faq.computersciencewiki.org/index.php/home/article/checksums>

Boost C++ Libraries. (s.f.). *CRC and checksum library*. Recuperado de <https://www.boost.org/doc/libs/release/libs/crc/crc.html>

Next.gr. (s.f.). *UART basics tutorial*. Recuperado de <https://www.next.gr/tutorials/digital-communication/uart-basics-tutorial>

University of Hawaii. (s.f.). *Error checking concepts*. Recuperado de <https://www2.hawaii.edu/~esb/1997fall.ics651/sep08.html>

Industrial Monitor Direct. (s.f.). *XOR checksum calculation for RS485 systems*. Recuperado de <https://industrialmonitordirect.com/blogs/knowledgebase/compactlogix-ascii-xor-checksum-calculation-for-rs485-display>

DigiKey. (s.f.). *Considerations for sending data over a wireless link*. Recuperado de <https://www.digikey.com/en/articles/considerations-for-sending-data-over-a-wireless-link>



